CodeIgniter
V3 框架中文文档

编　写　人：	翻译
日　　期：	2025-11-26
版　　本：	V1.0.0

修改历史

日期	版本	作者	修改内容
2025-11-26	1.0.01	翻译	初始版本

目录

第一章 序言 15

1.1. 欢迎使用 CodeIgniter 15

1.1.1. CodeIgniter 是为谁准备的 15

1.1.2. 版本号 15

1.2. 安装说明 16

1.2.1. 下载 CodeIgniter 16

1.2.2. 安装说明 17

1.2.3. 从老版本升级 18

1.2.3.1. Upgrading from 3.1.4 to 3.1.5 18

1.2.4. 疑难解答 18

1.3. 向 CodeIgniter 贡献你的力量 18

1.3.1. 支持 19

1.3.2. 安全 19

1.3.3. 提交好问题的技巧 19

1.3.4. 指导手册 20

1.3.5. PHP 代码规范 20

1.3.6. 文档 20

1.3.7. 兼容性 21

1.3.8. 分支 21

1.3.9. 签名 21

第二章 概览 22

2.1. 开始 22

2.2. CodeIgniter 是什么 22

2.3. 支持特性 24
2.4. 应用程序流程器 26
2.5. 模型-视图-控制器 27
2.6. 架构目标 27
第三章 教程-内容提要 28
3.1. 加载静态内容 29
3.1.1. 在控制器中添加逻辑 31
3.1.2. 路由 32
3.2. 读取新闻条目 33
3.2.1. 创建你的数据模型 33
3.2.2. 显示新闻 35
3.2.3. 路由 37
3.3. 创建新闻条目 38
3.3.1. 创建一个表单 38
3.3.2. 模型 38
3.3.3. 路由 39
3.4. 结束语 40
第四章 常规主题 40
4.1. CodeIgniter URL 40
4.1.1. URI 分段 41
4.1.2. 移除 URL 中的 index.php 41
4.1.3. 添加 URL 后缀 41
4.1.4. 启用查询字符串 42
4.2. 控制器 42

4.2.1. 什么是控制器 42
4.2.2. 让我们试试看：Hello World! 42
4.2.3. 方法 43
4.2.4. 通过 URI 分段向你的方法传递参数 44
4.2.5. 定义默认控制器 44
4.2.6. 重映射方法 44
4.2.7. 处理输出 44
4.2.8. 私有方法 44
4.2.9. 将控制器放入子目录中 44
4.2.10. 构造函数 44
4.2.11. 保留方法名 44
4.2.12. 就这样了 44
4.3. 保留名称 45
4.3.1. 控制器名称 45
4.3.2. 函数 45
4.3.3. 变量 46
4.3.4. 常量 46
4.4. 视图 47
4.4.1. 创建视图 47
4.4.2. 加载视图 47
4.5. 模型 48
4.5.1. 什么是模型 48
4.5.2. 剖析模型 49
4.5.3. 加载模型 49

4.5.4. 模型的自动加载 49
4.5.5. 连接数据库 50
4.6. 辅助函数 50
4.6.1. 加载辅助函数 50
4.6.2. 加载多个辅助函数 50
4.7. 使用 CodeIgniter 类库 51
4.8. 创建类库 51
4.9. 使用 CodeIgniter 驱动器 51
4.10. 创建驱动器 51
4.11. 创建核心系统类 52
4.12. 创建附属类 52
4.13. 钩子-扩展框架核心 52
4.14. 自动加载资源 52
4.15. 创建核心系统类 52
4.16. 创建核心系统类 52
第五章 类库参考 52
5.1. 基准测试类 52
5.1.1. 使用基准测试类 53
5.1.2. 类参考 53
5.2. 缓存驱动器 53
5.2.1. 使用示例 53
5.2.2. 类参考 54
5.2.2.1. is_supported($driver) 54
5.2.3. 驱动器 55

5.2.3.1. 可选 PHP 缓存（APC） 55

5.2.3.2. 基于文件的缓存 55

5.2.3.3. Memcached 缓存 55

5.2.3.4. Wincached 缓存 56

5.2.3.5. Redis 缓存 56

5.2.3.6. 虚拟缓存（Dummy Cache） 56

5.3. 日历类 56

5.3.1. 使用日历类 56

5.3.1.1. 初始化类 56

5.3.1.2. 显示一个日历 56

5.3.1.3. 传数据到单元格 57

5.3.1.4. 设置显示参数 57

5.4. 购物车类 57

5.5. 配置类 57

5.5.1. 使用配置类 58

5.5.1.1. 配置文件剖析 58

5.5.1.2. 加载配置文件 58

5.5.1.2.1. 手动加载 58

5.5.1.2.2. 自动加载 58

5.5.1.3. 获取配置项 58

5.5.1.4. 设置配置项 58

5.5.1.5. 多环境 59

5.6. Email 类 59

5.6.1. 使用 Email 类 59

5.6.1.1. 发送 Email 59
5.6.1.2. 设置 Email 60
5.6.1.3. Email 参数 60
5.6.1.4. 取消自动换行 60
5.7. 加密类 60
5.7.1. 使用加密类 60
5.7.1.1. 设置你的密钥 60
5.8. 文件上传类 60
5.8.1. 处理流程 60
5.8.1.1. 创建上传表单 61
5.8.1.2. 上传成功页面 62
5.8.1.3. 控制器 62
5.8.1.4. 上传文件目录 62
5.8.2. 参考指南 62
5.8.2.1. 初始化文件上传类 62
5.9. 图像处理类 62
5.9.1. 初始化类 62
5.9.1.1. 处理图像 62
5.9.1.2. 处理函数 63
5.9.1.3. 参数 63
5.9.1.4. 在配置文件中设置参数 64
5.9.2. 添加图像水印 64
5.9.2.1. 水印的两种类型 64
5.10. 分页类 64

5.10.1. 例子 64
5.10.1.1. 说明 64
5.10.1.2. 在配置文件中设置参数 65
5.10.2. 自定义分页 65
5.10.3. 添加封装标签 66
5.10.4. 自定义第一个链接 66
第六章 数据库参考 66
6.1. 数据库快速入门：示例代码 66
6.1.1. 初始化数据库类 66
6.1.2. 多结果查询（对象形式） 67
6.2. 数据库配置 67
6.2.1. 查询构造器 68
6.3. 连接数据库 68
6.3.1. 自动连接 68
6.3.2. 手动连接 69
6.3.2.1. 可用的参数 69
6.3.2.2. 手动连接到数据库 69
6.3.3. 连接到多个数据库 70
6.4. 查询 70
6.4.1. 基本查询 70
6.4.1.1. 常规查询 70
6.4.1.2. 简化查询 70
6.4.2. 指定数据库前缀 70
6.4.3. 保护标识符 71

6.4.4. 转义查询 71
6.4.5. 查询绑定 71
6.4.6. 错误处理 71
6.5. 生成查询结果 72
6.5.1. 结果数组 72
6.5.2. 结果行 72
6.5.3. 自定义结果对象 72
6.5.4. 结果辅助方法 72
6.5.5. Class Reference 72
6.6. 查询辅助函数 73
6.6.1. 关于执行查询的信息 73
6.6.2. 关于数据库的信息 73
6.7. 查询构造器类 73
6.7.1. 查询 73
6.7.1.1. 查询函数 1 73
6.7.1.2. 查询函数 2 74
6.7.2. 搜索 74
6.7.2.1. where 函数 1 74
6.7.2.2. where 函数 2 75
6.7.2.3. where 函数 3 75
6.7.3. 模糊搜索 75
6.7.4. 排序 75
6.7.5. 分页与计数 75
6.7.6. 查询条件组 75

6.7.7. 插入数据 75
6.7.8. 更新数据 75
6.7.9. 删除数据 76
6.7.10. 链式方法 76
6.7.11. 查询构造器缓存 76
6.7.12. 重置查询构造器 76
6.7.13. Class Reference 76
6.8. 事务 76
6.9. 获取元数据 76
6.10. 自定义函数调用 76
6.11. 查询缓存 76
6.12. 使用 Database Forge 维护数据库 76
6.13. 数据库工具类 76
6.14. 数据库驱动器参考 77
第七章 辅助函数参考 77
7.1. 数组辅助函数 77
7.1.1. 加载辅助函数 77
7.1.2. 可用函数 77
7.1.2.1. 函数 1 77
7.1.2.2. 函数 2 77
7.2. 验证码辅助函数 78
7.2.1. 加载辅助函数 78
7.2.2. 使用验证码辅助函数 78
7.2.3. 可用函数 78

7.3. Cookie 辅助函数 78
7.4. 日期辅助函数 78
7.5. 目录辅助函数 78
7.6. 下载辅助函数 78
7.7. 邮件辅助函数 78
7.7.1. 加载辅助函数 78
7.7.2. 可用函数 79
7.8. 文件辅助函数 79
7.9. 表单辅助函数 79
7.10. HTML 辅助函数 79
7.11. Inflector 辅助函数 79
7.12. 语言辅助函数 79
7.13. 数字辅助函数 79
7.14. 路径辅助函数 79
7.15. 安全辅助函数 79
7.16. 表情辅助函数 79
7.17. 字符串辅助函数 80
7.18. 文本辅助函数 80
7.19. 排版辅助函数 80
7.20. URL 辅助函数 80
7.21. XML 辅助函数 80
第八章 快速入门 80
8.1. 安装 80
8.1.1. 安装 80

8.1.2. 服务器要求 80
8.1.2.1. 安装 Laravel 81
8.1.2.2. 配置 81
8.1.3. Web 服务器配置 82
8.1.3.1. 目录配置 82
8.1.3.2. 优雅链接 82
8.2. 配置信息 82
8.2.1. 介绍 82
8.2.2. 环境配置 82
8.2.2.1. 环境变量类型 82
8.2.2.2. 检索环境配置 83
8.2.2.3. 确定当前环境 83
8.2.2.4. 在调试页面隐藏环境变量 84
8.2.3. 访问配置值 84
8.2.4. 配置缓存 84
8.2.5. 维护模式 84
8.3. 文件夹结构 84
8.3.1. 简介 84
8.3.2. 根目录 85
8.3.2.1. app 目录 85
8.3.2.2. 启动目录 85
8.3.2.3. 配置目录 85
8.3.2.4. 数据库目录 85
8.3.2.5. Public 公开目录 85

8.3.2.6. 资源目录 85
8.3.2.7. 路由目录 85
8.3.2.8. 文件存储目录 85
8.3.2.9. 测试目录 86
8.3.2.10. Vendor 目录 86
8.3.3. 应用目录 86
8.3.3.1. Broadcasting 目录 86
8.3.3.2. Console 目录 86
8.3.3.3. Events 目录 86
8.3.3.4. Exceptions 目录 86
8.3.3.5. Http 目录 86
8.3.3.6. Job 目录 86
8.3.3.7. Listeners 目录 87
8.3.3.8. Mail 目录 87
8.3.3.9. Notifications 目录 87
8.3.3.10. Policies 目录 87
8.3.3.11. Providers 目录 87
8.3.3.12. Rules 目录 87
8.4. 部署 87
8.4.1. 简介 87
8.4.2. 服务器配置 87
8.4.3. 改进 88
第九章 补充说明 88

第一章 序言

1.1. 欢迎使用 CodeIgniter

原文地址：https://codeigniter.com/userguide3/index.html

中文版地址：https://codeigniter.org.cn/userguide3/license.html

CodeIgniter 是一套给 PHP 网站开发者使用的应用程序开发框架和工具包。它的目标是让你能够更快速的开发，它提供了日常任务中所需的大量类库， 以及简单的接口和逻辑结构。通过减少代码量，CodeIgniter 让你更加专注 于你的创造性工作。

1.1.1. CodeIgniter 是为谁准备的

CodeIgniter 就是你所需要的，如果...

- 你想要一个小巧的框架；
- 你需要出色的性能；
- 你需要广泛兼容标准主机上的各种 PHP 版本和配置；
- 你想要一个几乎零配置的框架；
- 你想要一个不需使用命令行的框架；
- 你想要一个不想被编码规则的条条框框限制住的框架；
- 你对 PEAR 这种庞然大物不感兴趣；
- 你不想被迫学习一种新的模板语言（当然如果你喜欢，你可以选择一个模板解析器）；
- 你不喜欢复杂，追求简单；
- 你需要清晰、完整的文档。

1.1.2. 版本号

版本	发布时间	Bug 修复截止时间	安全修复截止时间
4.7.x	2026 年 3 月		
4.6.x	2025 年 3 月		

	14 日		
4.5.x	2025 年 9 月 7 日		
4.4.x	2023 年 12 月 28 日		
4.3.x	2023 年 1 月 14 日		
4.2.x	2022 年 9 月 6 日		
4.1.x	2021 年 9 月 6 月		
4.0.x	2020 年 2 月 14		
3	2015 年		

1.2. 安装说明

1.2.1. 下载 CodeIgniter

GitHub

Git 是一个分布式版本控制系统。

CodeIgniter 可以在 GitHub 上公开访问。请注意，尽管我们在保持代码的基础功能上做出了大量的努力，但是我们并不能为开发分支的代码中的功能作担保。

从 2.0.3 版本开始，每个版本的稳定版都可以从 GitHub Releases 获取。

1.2.2. 安装说明

通过下面四步来安装 CodeIgniter：

1.解压缩安装包；

2.将 CodeIgniter 文件夹及里面的文件上传到服务器，通常 index.php 文件将位于网站的根目录；

3.使用文本编辑器打开 application/config/config.php 文件设置你网站的根 URL，如果你想使用加密或会话，在这里设置上你的加密密钥；

4.如果你打算使用数据库，打开 application/config/database.php 文件设置数据库参数。

如果你想通过隐藏 CodeIgniter 的文件位置来增加安全性，你可以将 system 和 application 目录修改为其他的名字，然后打开主目录下的 index.php 文件将 $system_path 和 $application_folder 两个变量设置为你修改的名字。

为了达到更好的安全性，system 和 application 目录都应该放置在 Web 根目录之外，这样它们就不能通过浏览器直接访问。CodeIgniter 默认在每个目录下都包含了一个 .htaccess 文件，用于阻止直接访问，但是最好还是将它们移出能公开访问的地方，防止出现 Web 服务器配置更改或者 .htaccess 文件不被支持这些情况。

如果你想让 views 目录保持公开，也可以将你的 views 目录移出 application 目录。

移动完目录之后，打开 index.php 文件，分别设置好 $system_path 、$application_folder 和 $view_folder 三个变量的值，最好设置成绝对路径，例如：“/www/MyUser/system”。

在生产环境还要额外再多一步，就是禁用 PHP 错误报告以及所有其他仅在开发环境使用的功能。在 CodeIgniter 中，可以通过设置 ENVIRONMENT 常量来做到这一点，这在 安全 这篇指南中有着更详细的介绍。

以上就是全部安装过程！

1.2.3. 从老版本升级

请根据你要升级的版本阅读相应的升级注意事项。

1.2.3.1. Upgrading from 3.1.4 to 3.1.5

Before performing an update you should take your site offline by replacing the index.php file with a static one.

Step 1: Update your CodeIgniter files

Replace all files and directories in your system/ directory.

1.2.4. 疑难解答

如果你发现无论输入什么 URL 都只显示默认页面的话，那么可能是你的服务器不支持 PATH_INFO 变量，该变量用来提供搜索引擎友好的 URL 。 解决这个问题的第一步是打开 application/config/config.php 文件， 找到 URI Protocol 信息，根据注释提示，该值可以有几种不同的设置方式， 你可以逐个尝试一下。 如果还是不起作用，你需要让 CodeIgniter 强制在你的 URL 中添加一个问号， 要做到这点，你可以打开 application/config/config.php 文件， 然后将下面的代码:

```
$config['index_page'] = "index.php";
```

修改为这样:

```
$config['index_page'] = "index.php?";
```

1.3. 向 CodeIgniter 贡献你的力量

CodeIgniter 是一个社区驱动的项目，它会接受从社区里贡献的文档和代码。这些贡献都是通过 GitHub 上 CodeIgniter 代码仓库 的 Issues 或者 Pull Requests 的形式来完成的。

Issues 是一种快速提交 bug 的方式，如果你发现了一个 CodeIgniter 的 bug 或文档错误，那么请先 看看下面这几点：

- 是否已经存在一个打开的 Issue
- 该 Issue 是否已经被修复了（检查下 delevop 分支的代码，或者已关闭的 Issues）
- 这个 bug 很明显你一个人就能修复吗？

提交 Issues 是非常有用的，但是更好的做法是发起一个 Pull Request ，具体的做法是先 fork 主仓库的代码，然后将修改的代码提交到你自己的副本中。这需要你会使用版本控制系统 Git 。

1.3.1. 支持

请注意，GitHub 并不是用来回答一般的技术支持类问题的！

If you are having trouble using a feature of CodeIgniter, ask for help on our forums instead.

If you are not sure whether you are using something correctly or if you have found a bug, again - please ask on the forums first.

1.3.2. 安全

Did you find a security issue in CodeIgniter?

Please don't disclose it publicly, but e-mail us at security@codeigniter.com, or report it via our page on HackerOne.

If you've found a critical vulnerability, we'd be happy to credit you in our ChangeLog <../changelog>.

1.3.3. 提交好问题的技巧

使用描述性的标题（如：解析器类在处理逗号时出错），而不是使用模糊不清的标题（如：我的代码出错了）

在一份报告中只提交一个问题。

在问题中指出 CodeIgniter 的版本（如：3.0-develop），以及出问题的组件（如果你知道的话）（如：解析器类）

解释清楚你希望出现什么结果，以及当前出现的结果是什么。 如果有错误信息的话，并附上错误信息和堆栈信息。

如果有助于阐述你的问题的话，你可以包含少量的代码片段。 如果有大量的代码或截图的话，可以使用类似于 pastebin 或者 dropbox 这样的服务，不要在问题报告中包含这些内容。 为这些内容设置一个合理的过期时间，至少在问题被解决或关闭之前确保它们能访问。

如果你知道如何修复该问题，你可以 fork 并在你自己的分支中修改，然后提交一个 pull request 。 并将上面说的问题报告作为 pull request 的一部分。

如果能在问题报告中描述问题重现的详细步骤，那将是极好的。 如果你还能提供一个单元测试用例来重现该错误，那将更好，因为这给了修复这个问题的人一个更清晰的目标。

1.3.4. 指导手册

这里是如何提交 Pull Requests 的一些指南，如果你提交的 Pull Requests 没有遵循这篇指南中提出的这些， 你的提交可能会被拒绝并要求你重新提交。这可能听起来有点难，但是为了保证我们的代码质量这是必须要做的。

1.3.5. PHP 代码规范

所有的代码都必须符合 代码规范指南，它其实就是 Allman 缩进风格 加上下划线规则以及可读的操作符。遵循代码规则可以让代码的风格保持一致，同时也意味着代的更可读性更好。

1.3.6. 文档

如果你的修改同时也需要在文档中另加说明，那么你也需要在文档中加上它。新的类、方法、参数、默认值的修改 等等这些都需要对文档做相应的调整。每一处修改也必须要在代码的变更日志（change-log）中进行更新。另外，PHP 的文档注释块（PHPDoc blocks）也要修改。

1.3.7. 兼容性

CodeIgniter 推荐使用 PHP 5.6 或更高的版本，但是同时它也对 PHP 5.3.7 保持兼容，所以所有提交的代码都必须 满足这一点。如果你用到了 PHP 5.4（或以上版本）中的函数或新特性，这些代码需要回退到 PHP 5.3.7 版本的。

1.3.8. 分支

CodeIgniter 使用了 Git-Flow 分支模型， 这要求所有的 pull request 应该提交到 develop 分支，develop 分支是正在开发的打算在下一版发布的分支，master 分支总是包含最新的稳定版并保持干净，这样可以在例如出现紧急安全漏洞时快速的在 master 分支代码 上打上补丁并发布新的版本，而无需担心新加的功能会影响它。正是因为这个原因，所有的提交都应该在 develop 分支，发送到 master 分支的提交会被自动关闭。如果你的提交中包含多处修改，请将每一个修改都放到你独立的分支中。

一次只做一件事：一个 pull request 应该只包含一个修改。这不是意味着说一次提交，而是一次修改（尽管大多数时候 一次提交就是一次修改）。这样做的原因是如果你在同一个 pull request 中修改了 X 和 Y ，但是我们希望能合并 X 同时不想合并 Y ，这时我们就无法合并你的请求。你可以使用 Git-Flow 分支模型为每一个功能创建一个独立的分支， 然后提交两个请求。

1.3.9. 签名

你必须对你的工作进行签名，保证这些工作是你原创的或者不是你原创的但是你有将它们添加到开源项目中的权利。 在 Git 中签名并没有得到足够重视，所以你几乎用不到 --signoff 参数，但是在你提交代码到 CodeIgniter 时， 必须使用该参数。你必须对你的工作进行签名，保证这些工作是你原创的或者不是你原创的但是你有将它们添加到开源项目中的权利。 在 Git 中签名并没有得到足够重视，所以你几乎用不到 --signoff 参数，但是在你提交代码到 CodeIgniter 时，必须使用该参数。

```
git commit --signoff
```

或简写成：

```
git commit -s
```

这个命令会根据你 git 的配置信息在你的提交中添加签名。

第二章 概览

2.1. 开始

任何应用程序的学习都需要花费一定的精力。我们尽自己最大的努力使大家在学习中少走弯路，并且充满乐趣。

第一步就是 安装 CodeIgniter，然后阅读目录中介绍部分的所有主题。

接下来，按顺序阅读 常规主题 里面的每一篇内容，那里每一篇的内容都是承接前一篇的， 并且提供了样例代码，你可以尝试一下。

一旦你掌握了这些基础概念，你就可以去看看 类库参考 和 辅助函数参考，去学习如何使用 CodeIgniter 原生的类库和辅助函数文件。

如果遇到了问题可以到我们的 开发者社区 寻找解答，另外， 也可以去我们的 维基页面，里面有其他用户分享的例子代码。

2.2. CodeIgniter 是什么

CodeIgniter 是一个应用程序框架

CodeIgniter 是为 PHP 开发人员提供的一套 Web 应用程序工具包。 它的目标是能够让你比从零开始更加快速的完成项目，它提供了一套 丰富的的类库来满足我们日常的任务需求，并且提供了一个简单的 接口和逻辑结构来调用这些库。 CodeIgniter 通过最小化你需要的代码量， 让你把更多的精力放到项目的创造性开发上。

CodeIgniter 是免费的

CodeIgniter 通过 MIT 开源许可协议授权，你可以任意使用。阅读 许可协议 了解更多内容。

CodeIgniter 是轻量级的

真正的轻量级。我们的核心系统只需要一些非常小的库，这和那些 需要大量资源的框架完全相反。而且，库都是根据请求动态加载， 你需要什么才加载什么，所以核心系统是个非常轻快的系统。

CodeIgniter 是快速的

真正的快速。我们保证你很难找到一个比 CodeIgniter 性能更好的框架。

CodeIgniter 使用 M-V-C 架构

CodeIgniter 使用了模型-视图-控制器 架构，它能很好的将逻辑层和表示层分离。特别是对于那些使用了模板文件的项目来说更好，它能减少模板文件中的代码量。我们在介绍 MVC 架构的页面有更详细的介绍。

CodeIgniter 生成干净的 URL

CodeIgniter 生成的 URL 非常干净，而且对搜索引擎友好。不同于标准的 “查询字符串” 方法， CodeIgniter 使用了基于段的方法:

```
example.com/news/article/345
```

CodeIgniter 功能强大

CodeIgniter 拥有全面的类库，能满足大多数 Web 开发任务的需要， 例如：访问数据库，发送邮件，验证表单数据，会话管理，处理图像， 处理 XML-RPC 数据，等等等等。

CodeIgniter 是可扩展的

系统可以非常简单的通过你自己的类库和辅助函数来扩展，也可以通过 类扩展或系统钩子来实现。

CodeIgniter 不需要模板引擎

尽管 CodeIgniter 自带了一个非常简单的可选的模板解析器，但并不强制你 使用模板。模板引擎的性能无法和原生的 PHP 代码相比，另外使用模板引擎 还需要学习一种新的语法，而使用原生 PHP 代码只需要你掌握基本的 PHP 语法即可。看下面的 PHP 代码:

```
<ul>
<?php foreach ($addressbook as $name):?>
    <li><?=$name?></li>
<?php endforeach; ?>
</ul>
```

再来看看使用模板引擎的代码（伪代码）：

```
<ul>
{foreach from=$addressbook item="name"}
    <li>{$name}</li>
{/foreach}
</ul>
```

的确，模板引擎中的代码要清晰一点，但是它是以性能为代价的， 因为模板引擎中的伪代码必须要转换回 PHP 代码才能运行。 我们的一个目标是： 性能最大化 ，所以，我们决定不使用模板引擎。

CodeIgniter 拥有全面的文档

程序员们都喜欢写代码而讨厌写文档，当然，我们也不例外。 但是由于文档和代码是一样重要的，所以我们尽力来做好它。 我们的代码非常简洁并且注释也非常全面。

CodeIgniter 拥有一个友好的用户社区

你可以看到我们的社区用户在不断的增长，在 社区论坛 里活跃的参与着各种话题。

2.3. 支持特性

通过一个程序支持的特性并不能断定一个程序的好坏，因为特性不能体现出它良好的用户体验，不能体现出它优雅的设计，也不能显示出它代码质量如何，性能如何，细节处理的如何，安全性如何。判断程序好坏唯一的方法是体验一下并深入研究下它的代码。

安装 CodeIgniter 非常简单，所以我们鼓励你去安装它，同时，这里是一份 CodeIgniter 支持的主要特性列表。

基于 MVC 体系

超轻量级

对数种数据库平台的全特性支持

支持查询构造器

表单与数据验证

安全性与 XSS 过滤

会话管理

发送邮件类，支持附件、HTML 或文本邮件、多协议（sendmail、SMTP、Mail）及更多

图像处理库（裁剪、缩放、旋转等），支持 GD、ImageMagick 和 NetPBM

文件上传类

FTP 类

本地化

分页

数据加密

基准测试

全页面缓存

错误日志

应用程序评测

日历类

User-Agent 类

Zip 编码类

模板引擎类

Trackback 类

XML-RPC 类

单元测试类

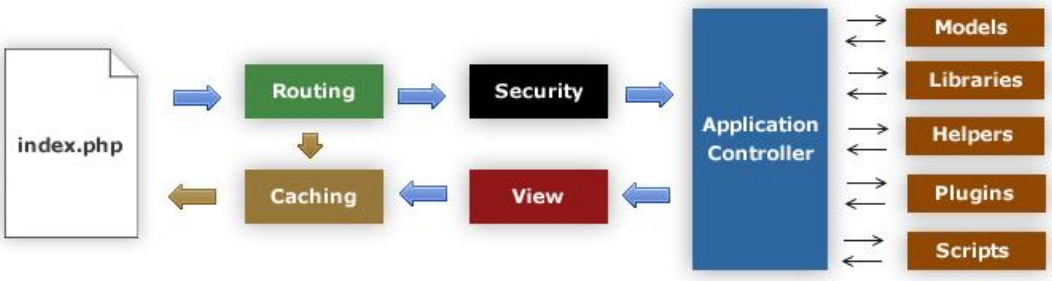
搜索引擎友好的 URL

灵活的 URL 路由

支持钩子和类扩展

大量的辅助函数

2.4. 应用程序流程器



1. index.php 文件作为前端控制器，初始化运行 CodeIgniter 所需的基本资源；
2. Router 检查 HTTP 请求，以确定如何处理该请求；
3. 如果存在缓存文件，将直接输出到浏览器，不用走下面正常的系统流程；
4. 在加载应用程序控制器之前，对 HTTP 请求以及任何用户提交的数据进行安全检查；
5. 控制器加载模型、核心类库、辅助函数以及其他所有处理请求所需的资源；

6. 最后一步，渲染视图并发送至浏览器，如果开启了缓存，视图被会先缓存起来用于 后续的请求。

2.5. 模型-视图-控制器

CodeIgniter 的开发基于 MVC（模型-视图-控制器）设计模式。MVC 是一种 用于将应用程序的逻辑层和表现层分离出来的软件方法。在实践中，由于这种分离 所以你的页面中只包含很少的 PHP 脚本。

模型 代表你的数据结构。通常来说，模型类将包含帮助你对数据库进行增删改查的方法。

视图 是要展现给用户的信息。一个视图通常就是一个网页，但是在 CodeIgniter 中，一个视图也可以是一部分页面（例如页头、页尾），它也可以是一个 RSS 页面，或其他任何类型的页面。

控制器 是模型、视图以及其他任何处理 HTTP 请求所必须的资源之间的中介，并生成网页。

CodeIgniter 在 MVC 使用上非常宽松，因为模型不是必需的。如果你不需要使用模型这一层，或是 认为维护模型比你想象中的复杂很多，那么你可以忽略它，而只使用控制器和视图创建你的应用程序。 CodeIgniter 可以让你直接调用你现有的脚本，甚至你也可以为系统开发核心类，总之， CodeIgniter 让你以你感觉最舒适的方式进行开发。

2.6. 架构目标

CodeIgniter 的目标是在最小化，最轻量级的开发包中得到最大的执行效率、功能和灵活性。

为了达到这个目标，我们在开发过程的每一步都致力于基准测试、重构和简化工作， 拒绝加入任何对实现目标没有帮助的东西。

从技术和架构角度看，CodeIgniter 按照下列目标创建：

1、动态实例化。 在 CodeIgniter 中，组件的导入和函数的执行都是在被请求的时候 才执行，而不是全局的。除核心资源外，系统不需要任何其他资源，因此系统默认是 非常轻量级的。HTTP 请求所触发的事件以及你设计的控制器和视图将决定哪些资源是 需要加载的。

2、松耦合。 耦合是指一个系统中组件之间的依赖程度。组件之间的依赖程度越低， 系统的重用性和灵活性就越好。我们的目标就是打造一个松耦合的系统。

3、组件专一性 专一性指的是组件对某个目标的专注程度。在 CodeIgniter 中，每一个类和方法都是高度独立的，从而可以最大程度的被复用。

CodeIgniter 是一个动态实例化，高度组件专一性的松耦合系统。它在小巧的基础上力求做到 简单、灵活和高性能。

第三章 教程-内容提要

通过由浅入深，前后连贯的章节安排，本教程将向你介绍 CodeIgniter 框架基础以及 MVC 程序架构的基本理论。

在这篇教程中，你将创建一个 简单新闻系统。首先，你将学会： 如何编写代码让程序能够加载静态内容页面；紧接着，你将学会： 从数据库中读取新闻条目并将其显示出来；最后，你将学会： 通过创建表单将新闻条目添加进数据库中。

教程将主要介绍以下技术要点：

- 模型-视图-控制器（Model-View-Controller）基础知识
- URI 路由基础
- 表单验证
- 使用 "查询构造器" 执行基本的数据库操作

整个教程会被分拆成多个章节，每个章节会介绍涉及到 CodeIgniter 的不同功能点。具体章节安排如下：

- 内容提要（本页面）：介绍教程将覆盖的内容要点。
- 加载静态内容：此节主要介绍控制器（Controllers）， 视图（Views）和路由（Routing）的基础知识。
- 读取新闻条目：此节开始介绍模型（Models）的相关知识， 以及在模型中执行一些基本的数据库操作。
- 创建新闻条目：此节主要介绍在 CodeIgniter 中执行高级数据库操作，以及表单验证的相关知识。
- 结束语：总结整个教程，给出深入学习 CodeIgniter 框架的一些建议和参考资源链接。

开始你的 CodeIgniter 框架探索之旅吧。

3.1. 加载静态内容

Note: 这篇教程假设你已经下载好 CodeIgniter ，并将其 安装 到你的开发环境。

你要做的第一件事情是新建一个控制器来处理静态页面，控制器就是一个简单的类，用来完成你的工作，它是你整个 Web 应用程序的 “粘合剂” 。

例如，当访问下面这个 URL 时: http://example.com/news/latest/10

通过这个 URL 我们就可以推测出来，有一个叫做 "news" 的控制器，被调用的方法为 "latest" ， 这个方法的作用应该是查询 10 条新闻条目并显示在页面上。在 MVC 模式里，你会经常看到下面 格式的 URL ：

http://example.com/[controller-class]/[controller-method]/[arguments]

在正式环境下 URL 的格式可能会更复杂，但是现在，我们只需要关心这些就够了。

新建一个文件 application/controllers/Pages.php ，然后添加如下代码。

```
<?php
class Pages extends CI_Controller {



    public function view($page = 'home')
```

```
    {

    }
}
```

你刚刚创建了一个 Pages 类，有一个方法 view 并可接受一个 $page 参数。 Pages 类继承自 CI_Controller 类，这意味着它可以访问 CI_Controller 类（ system/core/Controller.php ）中定义的方法和变量。

控制器将会成为你的 Web 应用程序中的处理请求的核心，在关于 CodeIgniter 的技术讨论中，这有时候被称作超级对象 。和其他的 PHP 类一样，可以在 你的控制器中使用 $this 来访问它，通过 $this 你就可以加载类库、 视图、以及针对框架的一般性操作。

现在，你已经创建了你的第一个方法，是时候创建一些基本的页面模板了，我们将 新建两个视图（页面模板）分别作为我们的页脚和页头。

新建页头文件 application/views/templates/header.php 并添加以下代码：

```
<html>
    <head>
        <title>CodeIgniter Tutorial</title>
    </head>
    <body>


        <h1><?php echo $title; ?></h1>
```

页头包含了一些基本的 HTML 代码，用于显示页面的主视图之前的内容。另外，它还打印出了 $title 变量，这个我们后面讲控制器的时候再讲。 现在，再新建个页脚文件 application/views/templates/footer.php ， 然后添加以下代码：

```
        <em>&copy; 2015</em>
```

```
    </body>
</html>
```

3.1.1. 在控制器中添加逻辑

你刚刚新建了一个控制器，里面有一个 view() 方法，这个方法接受一个参数 用于指定要加载的页面，静态页面模板位于 application/views/pages/ 目录。

在该目录中，再新建两个文件 home.php 和 about.php ，在每个文件里随便 写点东西然后保存它们。如果你没什么好写的，就写 "Hello World!" 吧。

为了加载这些页面，你需要先检查下请求的页面是否存在：

```
public function view($page = 'home')
{
  if ( ! file_exists(APPPATH.'views/pages/'.$page.'.php'))
    {
        // Whoops, we don't have a page for that!
        show_404();
    }

    $data['title'] = ucfirst($page); // Capitalize the first letter

    $this->load->view('templates/header', $data);
    $this->load->view('pages/'.$page, $data);
    $this->load->view('templates/footer', $data);
}
```

当请求的页面存在，将包括页面和页脚一起被加载并显示给用户，如果不存在， 会显示一个 "404 Page not found" 错误。

第一行检查页面是否存在，file_exists() 是个原生的 PHP 函数，用于检查某个 文件是否存在，show_404() 是个 CodeIgniter 内置的函数，用来显示一个默认的 错误页面。

在页头文件中，$title 变量用来自定义页面的标题，它是在这个方法中赋值的， 但是注意的是并不是直接赋值给 title 变量，而是赋值给一个 $data 数组的 title 元素。

最后要做的是按顺序加载所需的视图，view() 方法的第二个参数用于向视图传递参数， $data 数组中的每一项将被赋值给一个变量，这个变量的名字就是数组的键值。 所以控制器中 $data['title'] 的值，就等于视图中的 $title 的值。

3.1.2. 路由

控制器现在开始工作了！在你的浏览器中输入 [your-site-url]index.php/pages/view 来查看你的页面。当你访问 index.php/pages/view/about 时你将看到 about 页面， 包括页头和页脚。

使用自定义的路由规则，你可以将任意的 URI 映射到任意的控制器和方法上，从而打破 默认的规则：

```
http://example.com/[controller-class]/[controller-method]/[arguments]
```

让我们来试试。打开文件 application/config/routes.php 然后添加如下两行代码， 并删除掉其他对 $route 数组赋值的代码。

```
$route['default_controller'] = 'pages/view';

$route['(:any)'] = 'pages/view/$1';
```

CodeIgniter 从上到下读取路由规则并将请求映射到第一个匹配的规则，每一个规则都是 一个正则表达式（左侧）映射到 一个控制器和方法（右侧）。当有请求到来时，CodeIgniter 首先查找能匹配的第一条规则，然后调用相应的控

制器和方法，可能还带有参数。

你可以在关于 URI 路由的文档 中找到更多信息。

这里，第二条规则中 $routes 数组使用了通配符 (:any) 可以匹配所有的请求， 然后将参数传递给 Pages 类的 view() 方法。

现在访问 index.php/about 。路由规则是不是正确的将你带到了控制器中的 view() 方法？实在是太棒了！

3.2. 读取新闻条目

在上一节中，我们通过写出一个包含静态页面的类了解了一些框架的基本概念， 我们也根据自定义路由规则来重定向 URI 。现在是时候向大家介绍动态内容 和如何使用数据库了。

3.2.1. 创建你的数据模型

数据库的查询操作应该放在模型里，而不是写在控制器里，这样可以很方便的重用它。 模型正是用于从数据库或者其他存储中获取、新增、更新数据的地方。它就代表你的数据。

打开 application/models/ 目录，新建一个文件 News_model.php ，然后写入下面的代码。 确保你的 数据库配置 正确。

```
<?php
class News_model extends CI_Model {
    public function __construct()
    {
        $this->load->database();
    }
}
```

这个代码和之前的控制器的代码有点类似，它通过继承 *CI_Model* 创建了一个

新模型， 并加载了数据库类。数据库类可以通过 *$this->db* 对象访问。

在查询数据库之前，我们要先创建一个数据库表。连接你的数据库，运行下面的 SQL 语句（MySQL），并添加一些测试数据。

```
CREATE TABLE news (
    id int(11) NOT NULL AUTO_INCREMENT,
    title varchar(128) NOT NULL,
    slug varchar(128) NOT NULL,
    text text NOT NULL,
    PRIMARY KEY (id),
    KEY slug (slug)
);
```

现在，数据库和模型都准备好了，你需要一个方法来从数据库中获取所有的新闻文章。 为实现这点，我们使用了 CodeIgniter 的数据库抽象层 查询构造器 ， 通过它你可以编写你的查询代码，并在 所有支持的数据库平台 上运行。向你的模型中添加如下代码。

```
public function get_news($slug = FALSE)
{
    if ($slug === FALSE)
    {
        $query = $this->db->get('news');
        return $query->result_array();
    }
    $query = $this->db->get_where('news', array('slug' => $slug));
    return $query->row_array();
```

```
}
```

通过这个代码，你可以执行两种不同的查询，一种是获取所有的新闻条目，另一种 是根据它的 slug 来获取新闻条目。你应该注意到，$slug 变量在执行查询之前 并没有做检查， 查询构造器 会自动帮你检查的。

3.2.2. 显示新闻

现在，查询已经写好了，接下来我们需要将模型绑定到视图上，向用户显示新闻条目了。 这可以在之前写的 Pages 控制器里来做，但为了更清楚的阐述，我们定义了一个新的 News 控制器，创建在 application/controllers/News.php 文件中。

```
<?php
class News extends CI_Controller {


    public function __construct()
    {
        parent::__construct();
        $this->load->model('news_model');
        $this->load->helper('url_helper');
    }


    public function index()
    {
        $data['news'] = $this->news_model->get_news();
    }
```

```
    public function view($slug = NULL)
    {
        $data['news_item'] = $this->news_model->get_news($slug);
    }
}
```

阅读上面的代码你会发现，这和之前写的代码有些相似之处。首先是 __construct() 方法，它调用父类（CI_Controller）中的构造函数，并加载模型。这样模型就可以 在这个控制器的其他方法中使用了。另外它还加载了 URL 辅助函数 ， 因为我们在后面的视图中会用到它。

其次，有两个方法用来显示新闻条目，一个显示所有的，另一个显示特定的。你可以看到第二个方法中调用模型方法时传入了 $slug 参数，模型根据这个 slug 返回特定的新闻条目。

现在，通过模型，控制器已经获取到数据了，但还没有显示。下一步要做的就是， 将数据传递给视图。

```
public function index()
{
    $data['news'] = $this->news_model->get_news();
    $data['title'] = 'News archive';

    $this->load->view('templates/header', $data);
    $this->load->view('news/index', $data);
    $this->load->view('templates/footer');
}
```

上面的代码从模型中获取所有的新闻条目，并赋值给一个变量，另外页面的标题赋值给了 $data['title'] 元素，然后所有的数据被传递给视图。现在你需要创

建一个视图文件来 显示新闻条目了，新建 application/views/news/index.php 文件并添加如下代码。

```
<h2><?php echo $title; ?></h2>

<?php foreach ($news as $news_item): ?>

    <h3><?php echo $news_item['title']; ?></h3>
    <div class="main">
        <?php echo $news_item['text']; ?>
    </div>
    <p><a href="<?php echo
site_url('news/'.$news_item['slug']); ?>">View article</a></p>
<?php endforeach; ?>
```

这里，通过一个循环将所有的新闻条目显示给用户，你可以看到我们在 HTML 模板中混用了 PHP ，如果你希望使用一种模板语言，你可以使用 CodeIgniter 的 模板解析类 ，或其他的第三方解析器。

3.2.3. 路由

由于之前创建的通配符路由规则，你需要新增一条路由来显示你刚刚创建的控制器， 修改你的路由配置文件（application/config/routes.php）添加类似下面的代码。 该规则可以让请求访问 News 控制器而不是 Pages 控制器，第一行可以让 带 slug 的 URI 重定向到 News 控制器的 view() 方法。

```
$route['news/(:any)'] = 'news/view/$1';
$route['news'] = 'news';
$route['(:any)'] = 'pages/view/$1';
$route['default_controller'] = 'pages/view';
```

把浏览器的地址改回根目录，在后面加上 index.php/news 来看看你的新闻页面吧。

3.3. 创建新闻条目

现在你已经知道如何通过 CodeIgniter 从数据库中读取数据了，但是你还没有 向数据库中写入数据。在这一节，你将继续完善前文中创建的 News 控制器和模型， 添加上这一功能。

3.3.1. 创建一个表单

为了向数据库中写入数据，你需要先创建一个表单用来填写要存储的信息，这意味着 你的表单里需要包含两项：一项代表标题，另一项代表内容。你可以在模型中通过代码 从标题中提取出 slug 。 在文件 application/views/news/create.php 中创建一个新视图。

```
<h2><?php echo $title; ?></h2>

<?php echo validation_errors(); ?>

<?php echo form_open('news/create'); ?>

    <label for="title">Title</label>

    <input type="input" name="title" /><br />

    <label for="text">Text</label>

    <textarea name="text"></textarea><br />

    <input type="submit" name="submit" value="Create news item" />

</form>
```

这里有两个地方你可能还不熟悉：form_open() 函数 以及 validation_errors() 函数。

3.3.2. 模型

最后只剩下一件事情了，那就是写一个方法将数据保存到数据库中，你将会使用 输入类 获取用户提交的数据， 并使用 查询构造器类 向数据库中插入数

据。打开之前创建的模型文件，添加以下代码：

```
public function set_news()
{
    $this->load->helper('url');

    $slug = url_title($this->input->post('title'), 'dash', TRUE);

    $data = array(
        'title' => $this->input->post('title'),
        'slug' => $slug,
        'text' => $this->input->post('text')
    );

    return $this->db->insert('news', $data);
}
```

新加的这个方法用于向数据库插入数据，第三行有一个新方法 url_title() ，这个方法由 URL 辅助函数 提供，用于将字符串 中的所有空格替换成连接符（-），并将所有字符转换为小写。 这样其实就生成了一个 slug ，可以很好的用于创建 URI 。

3.3.3. 路由

在你开始向 CodeIgniter 程序中添加新闻条目之前，你需要到 config/routes.php 文件中去添加一条新的路由规则，确保你的文件中包含了下面的代码。这样可以让 CodeIgniter 知道 'create' 将作为一个方法被调用，而不是一个新闻条目的 slug。

```
$route['news/create'] = 'news/create';
```

```
$route['news/(:any)'] = 'news/view/$1';

$route['news'] = 'news';

$route['(:any)'] = 'pages/view/$1';

$route['default_controller'] = 'pages/view';
```

现在在你的浏览器中输入你安装好的 CodeIgniter 的本地开发地址，然后在 URL 后面添加上 index.php/news/create 。恭喜你，你刚刚完成了你的第一个 CodeIgniter 程序！添加些新闻来看看这些你创造的页面吧！

3.4. 结束语

这篇教程可能并非如你所愿，希望能涵盖一个内容管理系统的方方面面， 但是它至少带你了解了一些最为关键的知识点，包括路由、控制器、模型 等等。我们希望这篇教程可以帮助你了解一些 CodeIgniter 的基本设计模式， 在这个基础上你可以做更为深入的研究。

现在，你已经完成了这篇教程，我们推荐你再继续看看文档的其余部分。CodeIgniter 常常以文档全面而被称赞，你应该使用这个优势，通篇阅读“概览”和“常规主题”这两个部分，当你需要时查阅类库和辅助函数参考。

任何一个中级 PHP 程序员应该都可以在几天内的时间里上手 CodeIgniter。

如果你对框架或者你自己的 CodeIgniter 代码仍有疑问，你可以：

- 访问我们的 社区论坛
- 或访问我们的 IRC 聊天室
- 或浏览我们的 Wiki

第四章 常规主题

4.1. CodeIgniter URL

默认情况下，CodeIgniter 中的 URL 被设计成对搜索引擎和人类友好。 不

同于使用标准的 “查询字符串” 方法，CodeIgniter 使用基于段的方法:

```
example.com/news/article/my_article
```

4.1.1. URI 分段

如果遵循模型-视图-控制器模式，那么 URI 中的每一段通常表示下面的含义:

```
example.com/class/function/ID
```

1. 第一段表示要调用的控制器 类；
2. 第二段表示要调用的类中的 函数 或 方法；
3. 第三段以及后面的段代表传给控制器的参数，如 ID 或其他任何变量;

URI 类 和 URL 辅助函数 包含了一些函数可以让你更容易的处理 URI 数据，另外，你的 URL 可以通过 URI 路由 进行重定向从而得到更大的灵活性。

4.1.2. 移除 URL 中的 index.php

默认情况，你的 URL 中会包含 index.php 文件:

```
example.com/index.php/news/article/my_article
```

如果你的 Apache 服务器启用了 mod_rewrite ，你可以简单的通过一个 .htaccess 文件再加上一些简单的规则就可以移除 index.php 了。如果你的 Apache 服务器启用了 mod_rewrite ，你可以简单的通过一个 .htaccess 文件再加上一些简单的规则就可以移除 index.php 了。

4.1.3. 添加 URL 后缀

在你的 config/config.php 文件中你可以指定一个后缀，CodeIgniter 生成 URL 时会自动添加上它。例如，一个像这样的 URL:

```
example.com/index.php/products/view/shoes
```

你可以添加一个后缀，如：.html ，这样页面看起来就是这个样子:

```
example.com/index.php/products/view/shoes.html
```

4.1.4. 启用查询字符串

有些时候，你可能更喜欢使用查询字符串格式的 URL:

```
index.php?c=products&m=view&id=345
```

CodeIgniter 也支持这个格式，你可以在 application/config.php 配置文件中启用它。 打开你的配置文件，查找下面这几项:

```
$config['enable_query_strings'] = FALSE;
$config['controller_trigger'] = 'c';
$config['function_trigger'] = 'm';
```

你只要把 "enable_query_strings" 参数设为 TRUE 即可启用该功能。然后通过你设置的 trigger 关键字来访问你的控制器和方法:

```
index.php?c=controller&m=method
```

4.2. 控制器

控制器是你整个应用的核心，因为它们决定了 HTTP 请求将被如何处理。

4.2.1. 什么是控制器

简而言之，一个控制器就是一个类文件，是以一种能够和 URI 关联在一起的方式来命名的。

考虑下面的 URI:

```
example.com/index.php/blog/
```

上例中，CodeIgniter 将会尝试查询一个名为 Blog.php 的控制器并加载它。

4.2.2. 让我们试试看：Hello World!

接下来你会看到如何创建一个简单的控制器，打开你的文本编辑器，新建一个文件 Blog.php ， 然后放入以下代码:

```
<?phpclass Blog extends CI_Controller {


    public function index()
    {
        echo 'Hello World!';
    }}
```

4.2.3. 方法

URI 中的第二段用于决定调用控制器中的哪个方法。让我们试一下，向你的控制器添加一个新的方法:

```
<?phpclass Blog extends CI_Controller {


    public function index()
    {
        echo 'Hello World!';
    }


    public function comments()
    {
        echo 'Look at this!';
    }}
```

现在，通过下面的 URL 来调用 comments 方法:

```
example.com/index.php/blog/comments/
```

4.2.4. 通过 URI 分段向你的方法传递参数

如果你的 URI 多于两个段，多余的段将作为参数传递到你的方法中。

4.2.5. 定义默认控制器

CodeIgniter 可以设置一个默认的控制器，当 URI 没有分段参数时加载，例如当用户直接访问你网站的首页时。 打开 application/config/routes.php 文件，通过下面的参数指定一个默认的控制器:

```
$route['default_controller'] = 'blog';
```

4.2.6. 重映射方法

4.2.7. 处理输出

4.2.8. 私有方法

4.2.9. 将控制器放入子目录中

4.2.10. 构造函数

4.2.11. 保留方法名

4.2.12. 就这样了

4.3. 保留名称

为了便于编程，CodeIgniter 使用了一些函数、方法、类 和 变量名来实现。因此，这些名称不能被开发者所使用，下面是不能使用的保留名称列表。

4.3.1. 控制器名称

因为你的控制器类将继承主程序控制器，所以你的方法命名一定不能和 主程序控制器类中的方法名相同，否则你的方法将会覆盖他们。 下面列出了已经保留的名称，请不要将你的控制器命名为这些：

- CI_Controller
- Default
- index

4.3.2. 函数

- is_php()
- is_really_writable()
- load_class()
- is_loaded()
- get_config()
- config_item()
- show_error()
- show_404()
- log_message()
- set_status_header()
- get_mimes()
- html_escape()
- remove_invisible_characters()
- is_https()
- function_usable()
- get_instance()
- _error_handler()
- _exception_handler()
- _stringify_attributes()

4.3.3. 变量

- $config
- $db
- $lang

4.3.4. 常量

- ENVIRONMENT
- FCPATH
- SELF
- BASEPATH
- APPPATH
- VIEWPATH
- CI_VERSION
- MB_ENABLED
- ICONV_ENABLED
- UTF8_ENABLED
- FILE_READ_MODE
- FILE_WRITE_MODE
- DIR_READ_MODE
- DIR_WRITE_MODE
- FOPEN_READ
- FOPEN_READ_WRITE
- FOPEN_WRITE_CREATE_DESTRUCTIVE
- FOPEN_READ_WRITE_CREATE_DESTRUCTIVE
- FOPEN_WRITE_CREATE
- FOPEN_READ_WRITE_CREATE
- FOPEN_WRITE_CREATE_STRICT
- FOPEN_READ_WRITE_CREATE_STRICT
- SHOW_DEBUG_BACKTRACE
- EXIT_SUCCESS
- EXIT_ERROR
- EXIT_CONFIG
- EXIT_UNKNOWN_FILE
- EXIT_UNKNOWN_CLASS
- EXIT_UNKNOWN_METHOD
- EXIT_USER_INPUT

- EXIT_DATABASE
- EXIT__AUTO_MIN
- EXIT__AUTO_MAX

4.4. 视图

简单来说，一个视图其实就是一个 Web 页面，或者页面的一部分，像页头、页脚、侧边栏等。 实际上，视图可以很灵活的嵌在另一个视图里，然后这个视图再嵌在另一个视图里，等等， 如果你想使用这种层次结构的话，可以这样做。

视图不是直接被调用的，它必须通过 控制器 来加载。在 MVC 框架里，控制器扮演着类似于交警的角色，它专门负责读取特定的视图。如果你还没有读过控制器 页面，你应该先看下这个。

4.4.1. 创建视图

使用你的文本编辑器，创建一个 blogview.php 文件，代码如下:

```
<html><head>
    <title>My Blog</title></head>
<body>
    <h1>Welcome to my Blog!</h1>
</body>
</html>
```

4.4.2. 加载视图

使用下面的方法来加载指定的视图:

```
$this->load->view('name');
```

4.5. 模型

模型对于那些想使用更传统的 MVC 模式的人来说是可选的。

4.5.1. 什么是模型

模型是专门用来和数据库打交道的 PHP 类。例如，假设你使用 CodeIgniter 管理一个博客，那么你应该会有一个用于插入、更新以及获取博客数据的模型类。这里是一个模型类的例子:

```
class Blog_model extends CI_Model {


    public $title;
    public $content;
    public $date;


    public function get_last_ten_entries()
    {
        $query = $this->db->get('entries', 10);
        return $query->result();
    }
    public function insert_entry()
    {
        $this->title    = $_POST['title']; // please read the below
note
        $this->content  = $_POST['content'];
        $this->date = time();
```

```
        $this->db->insert('entries', $this);
    }

    public function update_entry()
    {
        $this->title    = $_POST['title'];
        $this->content  = $_POST['content'];
        $this->date = time();
        $this->db->update('entries', $this, array('id' =>
$_POST['id']));
    }
}
```

4.5.2. 剖析模型

模型类位于你的 application/models/ 目录下，如果你愿意，也可以在里面创建子目录。

4.5.3. 加载模型

你的模型一般会在你的 控制器 的方法中加载并调用，你可以使用下面的方法来加载模型:

```
$this->load->model('model_name');
```

4.5.4. 模型的自动加载

如果你发现你有一个模型需要在整个应用程序中使用，你可以让 CodeIgniter 在系统初始化时自动加载它。打开 application/config/autoload.php 文

件， 并将该模型添加到 autoload 数组中。

4.5.5. 连接数据库

当模型加载之后，它 并不会 自动去连接你的数据库，下面是一些关于 数据库连接的选项：

你可以在控制器或模型中使用 标准的数据库方法 连接数据库。
你可以设置第三个参数为 TRUE 让模型在加载时自动连接数据库，会使用你的数据库配置文件中的配置:

```
$this->load->model('model_name', '', TRUE);
```

4.6. 辅助函数

辅助函数，顾名思义，是帮助我们完成特定任务的函数。每个辅助函数文件都是某一类 函数的集合。例如， URL 辅助函数 帮助我们创建链接，表单辅助函数**帮助 我们创建表单元素，**本文辅助函数 帮助我们处理文本的格式化，Cookie 辅助函数 帮助我们读取或设置 Cookie ，文件辅助函数 帮助我们处理文件，等等。

4.6.1. 加载辅助函数

可以使用下面的方法简单的加载辅助函数:

```
$this->load->helper('name');
```

4.6.2. 加载多个辅助函数

如果你需要加载多个辅助函数，你可以使用一个数组，像下面这样:

```
$this->load->helper(
    array('helper1', 'helper2', 'helper3'));
```

4.7. 使用 CodeIgniter 类库

所有的系统类库都位于 system/libraries/ 目录下，大多数情况下，在使用之前， 你要先在 控制器 中初始化它，使用下面的方法:

```
$this->load->library('class_name');
```

4.8. 创建类库

当我们使用“类库”这个词的时候，通常我们指的是位于 libraries 这个目录下的那些类， 在我们这份用户手册的类库参考部分有详细的介绍。但是在这篇文章中，我们将介绍 如何在 application/libraries 目录下创建你自己的类库，和全局的框架类库独立开来。

4.9. 使用 CodeIgniter 驱动器

驱动器是一种特殊类型的类库，它有一个父类和任意多个子类。子类可以访问父类， 但不能访问兄弟类。在你的 控制器 中，驱动器为你的类库提供了 一种优雅的语法，从而不用将它们拆成很多离散的类。

驱动器位于 system/libraries/ 目录，每个驱动器都有一个独立的目录，目录名和 驱动器父类的类名一致，在该目录下还有一个子目录，命名为 drivers，用于存放 所有子类的文件。

要使用一个驱动器，你可以在控制器中使用下面的方法来进行初始化:

```
$this->load->driver('class_name');
```

4.10. 创建驱动器

下面是驱动器目录和文件结构布局的简单例子:

- /application/libraries/Driver_name
- Driver_name.php
- drivers

- Driver_name_subclass_1.php
- Driver_name_subclass_2.php
- Driver_name_subclass_3.php

4.11. 创建核心系统类

4.12. 创建附属类

4.13. 钩子-扩展框架核心

4.14. 自动加载资源

4.15. 创建核心系统类

4.16. 创建核心系统类

第五章 类库参考

5.1. 基准测试类

CodeIgniter 有一个一直都是启用状态的基准测试类，用于计算两个标记点之间的时间差。

另外，基准测试总是在框架被调用的那一刻开始，在输出类向浏览器发送最终的视图之前结束。 这样可以显示出整个系统执行的精确时间。

5.1.1. 使用基准测试类

基准测试类可以在你的 控制器、视图 以及 模型 中使用。

5.1.2. 类参考

基准测试类可以在你的 控制器、视图 以及 模型 中使用。

5.2. 缓存驱动器

CodeIgniter 提供了几种最常用的快速缓存的封装，除了基于文件的缓存，其他的缓存都需要对服务器进行特殊的配置，如果配置不正确，将会抛出 一个致命错误异常（Fatal Exception）。

5.2.1. 使用示例

下面的示例代码用于加载缓存驱动器，使用 APC 作为缓存，如果 APC 在服务器环境下不可用，将降级到基于文件的缓存。

```
$this->load->driver('cache', array('adapter' => 'apc', 'backup' =>
'file'));


if ( ! $foo = $this->cache->get('foo'))
{
    echo 'Saving to the cache!<br />';
    $foo = 'foobarbaz!';

    // Save into the cache for 5 minutes
    $this->cache->save('foo', $foo, 300);
```

```
}


echo $foo;
```

你也可以设置 key_prefix 参数来给缓存名添加前缀，当你在同一个环境下运行多个应用时，它可以避免冲突。

```
$this->load->driver('cache',
    array('adapter' => 'apc', 'backup' => 'file', 'key_prefix' =>
'my_'));
$this->cache->get('foo'); // Will get the cache entry named 'my_foo'
```

5.2.2. 类参考

Class CI_Cache

5.2.2.1. is_supported($driver)

参数: $driver (string) -- the name of the caching driver

返回: TRUE if supported, FALSE if not

返回类型: bool

当使用 $this->cache->get() 方法来访问驱动器时该方法会被自动调用，但是，如果你使用了某些个人的驱动器， 应该先调用该方法确保这个驱动器在服务器环境下是否被支持。

```
if ($this->cache->apc->is_supported())
{
    if ($data = $this->cache->apc->get('my_cache'))
    {
        // do things.
```

```
    }
}
```

5.2.3. 驱动器

5.2.3.1. 可选 PHP 缓存（APC）

上述所有方法都可以直接使用，而不用在加载驱动器时指定 adapter 参数，如下所示:

```
$this->load->driver('cache');
$this->cache->apc->save('foo', 'bar', 10);
```

5.2.3.2. 基于文件的缓存

和输出类的缓存不同的是，基于文件的缓存支持只缓存视图的某一部分。使用这个缓存时要注意， 确保对你的应用程序进行基准测试，因为当磁盘 I/O 频繁时可能对缓存有负面影响。

上述所有方法都可以直接使用，而不用在加载驱动器时指定 adapter 参数，如下所示:

```
$this->load->driver('cache');
$this->cache->file->save('foo', 'bar', 10);
```

5.2.3.3. Memcached 缓存

可以在 memcached.php 配置文件中指定多个 Memcached 服务器，配置文件位于 application/config/ 目录。

上述所有方法都可以直接使用，而不用在加载驱动器时指定 adapter 参数，如下所示:

```
$this->load->driver('cache');
```

```
$this->cache->memcached->save('foo', 'bar', 10);
```

5.2.3.4. Wincached 缓存

5.2.3.5. Redis 缓存

5.2.3.6. 虚拟缓存（Dummy Cache）

5.3. 日历类

使用日历类可以让你动态的创建日历，并且可以使用日历模板来格式化显示你的日历， 允许你 100% 的控制它设计的每个方面。另外，你还可以向日历的单元格传递数据。

5.3.1. 使用日历类

5.3.1.1. 初始化类

跟 CodeIgniter 中的其他类一样，可以在你的控制器中使用 $this->load->library() 方法加载日历类:

```
$this->load->library('calendar');
```

5.3.1.2. 显示一个日历

这里是一个简单的例子，告诉你如何去显示一个日历:

```
$this->load->library('calendar');

echo $this->calendar->generate();
```

5.3.1.3. 传数据到单元格

如果需要添加数据到日历的单元格中，就要创建一个关联数组，这个数组中索引是想添加数据的日期， 数组对应的值是想添加的数据。该数组通过日历生成函数的第三个参数被传入，参考下面这个例子:

```
$this->load->library('calendar');

$data = array(

    3  => 'http://example.com/news/article/2006/06/03/',

    7  => 'http://example.com/news/article/2006/06/07/',

    13 => 'http://example.com/news/article/2006/06/13/',

    26 => 'http://example.com/news/article/2006/06/26/');

echo $this->calendar->generate(2006, 6, $data);
```

使用上面的例子，天数为 3、7、13 和 26 将变成链接指向你提供的 URL 。

5.3.1.4. 设置显示参数

有 8 种不同的参数可以让你设置日历的各个方面，你可以通过加载函数的第二个参数来设置参数。

5.4. 购物车类

购物车类允许项目被添加到 session 中，session 在用户浏览你的网站期间都保持有效状态。 这些项目能够以标准的 "购物车" 格式被检索和显示，并允许用户更新数量或者从购物车中移除项目。

5.5. 配置类

配置类用于获取配置参数，这些参数可以来自于默认的配置文件（application/config/config.php）， 也可以来自你自定义的配置文件。

5.5.1. 使用配置类

5.5.1.1. 配置文件剖析

CodeIgniter 默认有一个主要的配置文件，位于 application/config/config.php 。 如果你使用文本编辑器打开它的话，你会看到配置项都存储在一个叫做 $config 的数组中。

5.5.1.2. 加载配置文件

5.5.1.2.1. 手动加载

要加载你自定义的配置文件，你需要在 控制器 中使用下面的方法:

```
$this->config->load('filename');
```

5.5.1.2.2. 自动加载

如果你发现有一个配置文件你需要在全局范围内使用，你可以让系统自动加载它。要实现这点，打开位于 application/config/ 目录下的 autoload.php 文件，将你的配置文件添加到自动加载数组中。

5.5.1.3. 获取配置项

要从你的配置文件中获取某个配置项，使用如下方法:

```
$this->config->item('item_name');
```

其中 item_name 是你希望获取的 $config 数组的索引名，例如，要获取语言选项， 你可以这样:

```
$lang = $this->config->item('language');
```

5.5.1.4. 设置配置项

如果你想动态的设置一个配置项，或修改某个已存在的配置项，你可以这样:

```
$this->config->set_item('item_name', 'item_value');
```

5.5.1.5. 多环境

你可以根据当前的环境来加载不同的配置文件，index.php 文件中定义了 ENVIRONMENT 常量，在 处理多环境 中有更详细的介绍。

5.6. Email 类

CodeIgniter 拥有强大的 Email 类支持以下特性：

- 多协议：Mail、Sendmail 和 SMTP
- SMTP 协议支持 TLS 和 SSL 加密
- 多个收件人
- 抄送（CC）和密送（BCC）
- HTML 格式邮件 或 纯文本邮件
- 附件
- 自动换行
- 优先级
- 密送批处理模式（BCC Batch Mode），大邮件列表将被分成小批次密送
- Email 调试工具

5.6.1. 使用 Email 类

5.6.1.1. 发送 Email

发送邮件不仅很简单，而且你可以通过参数或通过配置文件设置发送邮件的不同选项。

下面是个简单的例子，用于演示如何发送邮件。注意：这个例子假设你是在某个 控制器 里面发送邮件。

```
$this->load->library('email');

$this->email->from('your@example.com', 'Your Name');$this->email->to('someone@example.com');$this->email->cc('another@another-example.com');$this->email->bcc('them@their-example.com');
```

```
$this->email->subject('Email Test');$this->email->message('Testing the email class.');

$this->email->send();
```

5.6.1.2. 设置 Email

有 21 种不同的参数可以用来对你发送的邮件进行配置。

5.6.1.3. Email 参数

5.6.1.4. 取消自动换行

5.7. 加密类

加密类提供了双向数据加密的方式，它依赖于 PHP 的 Mcrypt 扩展，所以要有 Mcrypt 扩展才能运行。

5.7.1. 使用加密类

5.7.1.1. 设置你的密钥

密钥 是对字符串进行加密或解密的一段信息片段。实际上，你设置的密钥是 唯一 能解密通过该密钥加密的数据，所以一定要慎重选择你的密钥， 而且如果你打算对持久数据进行加密的话，你最好不要修改密钥。

5.8. 文件上传类

CodeIgniter 的文件上传类用于上传文件，你可以设置参数限制上传文件的类型和大小。

5.8.1. 处理流程

上传一个文件通常涉及以下几步：

- 显示一个上传表单，用户选择文件并上传。
- 当提交表单时，文件将被上传到你指定的目录。
- 同时，根据你设置的参数对文件进行校验是否允许上传。
- 上传成功后，向用户显示成功消息。

5.8.1.1. 创建上传表单

使用文本编辑器新建一个文件 upload_form.php ，放入如下代码，并保存到 application/views/ 目录下:

```
<html>

<head>

<title>Upload Form</title>

</head>

<body>



<?php echo $error;?>

<?php echo form_open_multipart('upload/do_upload');?>

<input type="file" name="userfile" size="20" />

<br /><br />

<input type="submit" value="upload" />

</form>



</body>
```

你会注意到我们使用了表单辅助函数来创建 form 的起始标签，文件上传需要使用 multipart 表单， 辅助函数可以帮你正确生成它。还要注意的是，代码里有一个 $error 变量，当发生错误时， 可以用它来显示错误信息。

5.8.1.2. 上传成功页面

5.8.1.3. 控制器

5.8.1.4. 上传文件目录

5.8.2. 参考指南

5.8.2.1. 初始化文件上传类

5.9. 图像处理类

CodeIgniter 的图像处理类可以使你完成以下的操作：

- 调整图像大小
- 创建缩略图
- 图像裁剪
- 图像旋转
- 添加图像水印

可以很好的支持三个主流的图像库：GD/GD2、NetPBM 和 ImageMagick 。

5.9.1. 初始化类

跟 CodeIgniter 中的其他类一样，可以在你的控制器中使用 $this->load->library() 方法加载图像处理类:

```
$this->load->library('image_lib');
```

5.9.1.1. 处理图像

不管你想进行何种图像处理操作（调整大小，图像裁剪，图像旋转，添加水印）， 通常过程都是一样的。你会先设置一些你想进行的图像操作的参数， 然后调用四个可用方法中的一个。例如，创建一个图像缩略图:

```
$config['image_library'] = 'gd2';

$config['source_image'] = '/path/to/image/mypic.jpg';

$config['create_thumb'] = TRUE;

$config['maintain_ratio'] = TRUE;$config['width']     = 75;

$config['height']   = 50;

$this->load->library('image_lib', $config);

$this->image_lib->resize();
```

以上代码告诉 image_resize 函数去查找位于 source_image 目录下的名为 mypic.jpg 的图片,然后运用 GD2 图像库创建一个 75 X 50 像素的缩略图。当 maintain_ratio 选项设为 TRUE 时，生成的缩略图将保持图像的纵横比例，同时尽可能的在宽度和 高度上接近所设定的 width 和 height 。缩略图将被命名为类似 mypic_thumb.jpg 的形式，并保存在 source_image 的同级目录中。

5.9.1.2. 处理函数

有五个处理函数可以调用：

- $this->image_lib->resize()
- $this->image_lib->crop()
- $this->image_lib->rotate()
- $this->image_lib->watermark()

当调用成功时，这些函数会返回 TRUE ，否则会返回 FALSE 。 如果调用失败时，用以下函数可以获取错误信息:

```
echo $this->image_lib->display_errors();
```

5.9.1.3. 参数

你可以用下面的参数来对图像处理进行配置，满足你的要求。

5.9.1.4. 在配置文件中设置参数

如果你不喜欢使用上面的方法来设置参数，你可以将参数保存到配置文件中。你只需简单的创建一个文件 image_lib.php 并将 $config 数组放到该文件中，然后保存文件到 config/image_lib.php，这些参数将会自动被使用。 如果你在配置文件中设置参数，那么你就不需要使用 $this->image_lib->initialize() 方法了。

5.9.2. 添加图像水印

水印处理功能需要 GD/GD2 库的支持。

5.9.2.1. 水印的两种类型

你可以使用以下两种图像水印处理方式。

5.10. 分页类

CodeIgniter 的分页类非常容易使用，而且它 100% 可定制，可以通过动态的参数， 也可以通过保存在配置文件中的参数。

5.10.1. 例子

下面是一个简单的例子，如何在你的 控制器 方法中创建分页:

```
$this->load->library('pagination');

$config['base_url'] =
'http://example.com/index.php/test/page/';$config['total_rows'] =
200;$config['per_page'] = 20;

$this->pagination->initialize($config);

echo $this->pagination->create_links();
```

5.10.1.1. 说明

如上所示，$config 数组包含了你的配置参数，被传递

到 $this->pagination->initialize() 方法。 另外还有二十几个配置参数你可以选择，但是最少你只需要这三个配置参数。下面是这几个参数的含义：

- base_url 这是一个指向你的分页所在的控制器类/方法的完整的 URL ，在上面的这个例子里， 它指向了一个叫 "Test" 的控制器和它的一个叫 "Page" 的方法。记住，如果你需要一个不同格式的 URL ， 你可以 重新路由 。
- total_rows 这个数字表示你需要做分页的数据的总行数。通常这个数值是你查询数据库得到的数据总量。
- per_page 这个数字表示每个页面中希望展示的数量，在上面的那个例子中，每页显示 20 个项目。

当你没有分页需要显示时，*create_links()* 方法会返回一个空的字符串。

5.10.1.2. 在配置文件中设置参数

如果你不喜欢用以上的方法进行参数设置，你可以将参数保存到配置文件中。简单地创建一个名为 pagination.php 的文件，把 $config 数组加到这个文件中，然后将文件保存到 application/config/pagination.php 。这样它就可以自动被调用。用这个方法，你不再需要使用 $this->pagination->initialize 方法。

5.10.2. 自定义分页

下面是所有的参数列表，可以传递给 initialization 方法来定制你喜欢的显示效果。

$config['uri_segment'] = 3;

分页方法自动检测你 URI 的哪一段包含页数，如果你的情况不一样，你可以明确指定它。

$config['num_links'] = 2;

放在你当前页码的前面和后面的“数字”链接的数量。比方说值为 2 就会在每一边放置两个数字链接， 就像此页顶端的示例链接那样。

$config['use_page_numbers'] = TRUE;

默认分页的 URL 中显示的是你当前正在从哪条记录开始分页，如果你希望显示实际的页数，将该参数设置为 TRUE 。

```
$config['page_query_string'] = TRUE;
```

5.10.3. 添加封装标签

如果你希望在整个分页的周围用一些标签包起来，你可以通过下面这两个参数：

```
$config['full_tag_open'] = '<p>';
```

起始标签放在所有结果的左侧。

```
$config['full_tag_close'] = '</p>';
```

结束标签放在所有结果的右侧。

5.10.4. 自定义第一个链接

```
$config['first_link'] = 'First';
```

左边第一个链接显示的文本，如果你不想显示该链接，将其设置为 FALSE 。

第六章 数据库参考

6.1. 数据库快速入门：示例代码

6.1.1. 初始化数据库类

下面的代码将根据你的数据库配置加载并初始化数据库类：

```
$this->load->database();
```

数据库类一旦载入，你就可以像下面介绍的那样使用它。

6.1.2. 多结果查询（对象形式）

需要确认自己的环境满足以下要求：

```
$query = $this->db->query('SELECT name, title, email FROM my_table');

foreach ($query->result() as $row){

    echo $row->title;

    echo $row->name;

    echo $row->email;}

echo 'Total Results: ' . $query->num_rows();
```

上面的 result() 函数返回一个**对象数组**。例如：$row->title

6.2. 数据库配置

CodeIgniter 有一个配置文件用来保存数据库配置（用户名、密码、数据库名等等），这个配置文件位于 application/config/database.php。 You can also set database connection values for specific environments by placing database.php in the respective environment config folder.

配置存放在一个多维数组里，原型如下：

```
$db['default'] = array(

    'dsn'   => '',

    'hostname' => 'localhost',

    'username' => 'root',

    'password' => '',

    'database' => 'database_name',

    'dbdriver' => 'mysqli',

    'dbprefix' => '',
```

```
    'pconnect' => TRUE,
    'db_debug' => TRUE,
    'cache_on' => FALSE,
    'cachedir' => '',
    'char_set' => 'utf8',
    'dbcollat' => 'utf8_general_ci',
    'swap_pre' => '',
    'encrypt' => FALSE,
    'compress' => FALSE,
    'stricton' => FALSE,
    'failover' => array()
);
```

6.2.1. 查询构造器

可以通过数据库配置文件里的 $query_builder 变量对 查询构造器类 进行全局的设定（启用设成 TRUE，禁用设成 FALSE，默认是 TRUE）。如果你不用这个类，那么你可以通过将这个变量值设置成 FALSE 来减少在数据库类初始化时对电脑资源的消耗。

```
$query_builder = TRUE;
```

6.3. 连接数据库

6.3.1. 自动连接

“自动连接”特性将在每一个页面加载时自动实例化数据库类。要启用“自动

连接”， 可在 application/config/autoload.php 中的 library 数组里添加 database:

```
$autoload['libraries'] = array('database');
```

6.3.2. 手动连接

如果你只有一部分页面需要数据库连接，你可以在那些有需要的函数里手工添加 如下代码来连接数据库，或者写在类的构造函数里，让整个类都可以访问：

```
$this->load->database();
```

如果 database() 函数没有指定第一个参数，它将使用数据库配置文件中 指定的组连接数据库。对大多数人而言，这是首选方案。

6.3.2.1. 可用的参数

1. 数据库连接值，用数组或 DSN 字符串传递；
2. TRUE/FALSE (boolean) - 是否返回连接 ID（参考下文的“连接多数据库”）；
3. TRUE/FALSE (boolean) - 是否启用查询构造器类，默认为 TRUE 。

6.3.2.2. 手动连接到数据库

这个函数的第一个参数是**可选的**，被用来从你的配置文件中 指定一个特定的数据库组，甚至可以使用没有在配置文件中定义的 数据库连接值。下面是例子：

从你的配置文件中选择一个特定分组:

```
$this->load->database('group_name');
```

6.3.3. 连接到多个数据库

如果你需要同时连接到多个不同的数据库，可以这样:

```
$DB1 = $this->load->database('group_one', TRUE);

$DB2 = $this->load->database('group_two', TRUE);
```

6.4. 查询

6.4.1. 基本查询

6.4.1.1. 常规查询

要提交一个查询，使用 query 函数:

```
$this->db->query('YOUR QUERY HERE');
```

当你执行读类型的查询（如：SELECT）时，query() 函数将以**对象**形式返回一个结果集，参考这里来 显示你的结果。 当你执行写类型的查询（如：INSERT、DELETE、UPDATE）时，函数将简单的返回 TRUE 或 FALSE 来表示操作是否成功。 你可以将函数返回的结果赋值给一个变量，这样你就可以根据这个变量来获取 数据了，像下面这样:

```
$query = $this->db->query('YOUR QUERY HERE');
```

6.4.1.2. 简化查询

simple_query 函数是 $this->db->query() 的简化版。它不会返回查询的 结果集，不会去设置查询计数器，不会去编译绑定的数据，不会去存储查询的调试信息。 它只是用于简单的提交一个查询，大多数用户并不会用到这个函数。

6.4.2. 指定数据库前缀

如果你配置了一个数据库前缀参数，想把它加上你的 SQL 语句里的表名前面， 你可以调用下面的方法:

```
$this->db->dbprefix('tablename'); // outputs prefix_tablename
```

6.4.3. 保护标识符

在很多数据库里，保护表名和字段名是可取的，例如在 MySQL 数据库里使用反引号。使用查询构造器会自动保护标识符，尽管如此，你还是可以像下面这样手工来保护:

```
$this->db->protect_identifiers('table_name');
```

6.4.4. 转义查询

在提交数据到你的数据库之前，确保先对其进行转义是个非常不错的做法。

6.4.5. 查询绑定

查询绑定可以简化你的查询语法，它通过系统自动的为你将各个查询组装在一起。 参考下面的例子:

```
$sql = "SELECT * FROM some_table WHERE id = ? AND status = ? AND
author = ?";

$this->db->query($sql, array(3, 'live', 'Rick'));
```

6.4.6. 错误处理

```
$this->db->error();
```

要获取最近一次发生的错误，使用 *error()* 方法可以得到一个包含错误代码和错误消息的数组。 这里是一个简单例子:

```
if ( ! $this->db->simple_query('SELECT `example_field` FROM
`example_table`')){

    $error = $this->db->error(); // Has keys 'code' and 'message'}
```

6.5. 生成查询结果

6.5.1. 结果数组

result() 方法

该方法以**对象数组**形式返回查询结果，如果查询失败返回**空数组**。 一般情况下，你会像下面这样在一个 foreach 循环中使用它:

```
$query = $this->db->query("YOUR QUERY");

foreach ($query->result() as $row){

    echo $row->title;

    echo $row->name;

    echo $row->body;}
```

该方法是 *result_object()* 方法的别名。

6.5.2. 结果行

6.5.3. 自定义结果对象

6.5.4. 结果辅助方法

6.5.5. Class Reference

6.6. 查询辅助函数

6.6.1. 关于执行查询的信息

6.6.2. 关于数据库的信息

6.7. 查询构造器类

6.7.1. 查询

下面的方法用来构建 **SELECT** 语句。

$this->db->get()

该方法执行 SELECT 语句并返回查询结果，可以得到一个表的所有数据:

```
$query = $this->db->get('mytable');  // Produces: SELECT * FROM
mytable
```

第二和第三个参数用于设置 LIMIT 子句:

```
$query = $this->db->get('mytable', 10, 20);
```

你应该已经注意到了，上面的方法的结果都赋值给了一个 $query 变量，通过这个变量， 我们可以得到查询的结果:

```
$query = $this->db->get('mytable');
foreach ($query->result() as $row){
    echo $row->title;}
```

6.7.1.1. 查询函数 1

$this->db->get_compiled_select()

该方法和 $this->db->get() 方法一样编译 SELECT 查询并返回查询的 SQL 语句， 但是，该方法并不执行它。

6.7.1.2. 查询函数 2

$this->db->get_where()

这个方法基本上和上面的方法一样，但它提供了第二个参数可以让你添加一个 WHERE 子句， 而不是使用 *db->where()* 方法:

```
$query = $this->db->get_where('mytable', array('id' => $id), $limit, $offset);
```

6.7.2. 搜索

6.7.2.1. where 函数 1

$this->db->where()

该方法提供了 4 种方式让你编写查询语句中的 WHERE 子句：

1.简单的 key/value 方式:

```
$this->db->where('name', $name); // Produces: WHERE name = 'Joe'
```

2.自定义 key/value 方式:

为了控制比较，你可以在第一个参数中包含一个比较运算符：

```
$this->db->where('name !=', $name);

$this->db->where('id <', $id); // Produces: WHERE name != 'Joe' AND id < 45
```

3.关联数组方式:

```
$array = array('name' => $name, 'title' => $title, 'status' => $status);
```

```
$this->db->where($array);// Produces: WHERE name = 'Joe' AND title = 'boss' AND status = 'active'
```

4.自定义字符串:

你可以完全手工编写 WHERE 子句:

```
$where = "name='Joe' AND status='boss' OR status='active'";
$this->db->where($where);
```

6.7.2.2. where 函数 2

6.7.2.3. where 函数 3

6.7.3. 模糊搜索

6.7.4. 排序

6.7.5. 分页与计数

6.7.6. 查询条件组

6.7.7. 插入数据

6.7.8. 更新数据

6.7.9. 删除数据

6.7.10. 链式方法

6.7.11. 查询构造器缓存

6.7.12. 重置查询构造器

6.7.13. Class Reference

6.8. 事务

6.9. 获取元数据

6.10. 自定义函数调用

6.11. 查询缓存

6.12. 使用 Database Forge 维护数据库

6.13. 数据库工具类

6.14. 数据库驱动器参考

第七章 辅助函数参考

7.1. 数组辅助函数

7.1.1. 加载辅助函数

该辅助函数通过下面的代码加载:

```
$this->load->helper('array');
```

7.1.2. 可用函数

7.1.2.1. 函数 1

element($item, $array[, $default = NULL])

参数:

$item (string) -- Item to fetch from the array

$array (array) -- Input array

$default (bool) -- What to return if the array isn't valid

返回: NULL on failure or the array item.

返回类型: mixed

7.1.2.2. 函数 2

7.2. 验证码辅助函数

7.2.1. 加载辅助函数

该辅助函数通过下面的代码加载:

```
$this->load->helper('captcha');
```

7.2.2. 使用验证码辅助函数

7.2.3. 可用函数

7.3. Cookie 辅助函数

7.4. 日期辅助函数

7.5. 目录辅助函数

7.6. 下载辅助函数

7.7. 邮件辅助函数

7.7.1. 加载辅助函数

该辅助函数通过下面的代码加载:

```
$this->load->helper('email');
```

7.7.2. 可用函数

7.8. 文件辅助函数

7.9. 表单辅助函数

7.10. HTML 辅助函数

7.11. Inflector 辅助函数

7.12. 语言辅助函数

7.13. 数字辅助函数

7.14. 路径辅助函数

7.15. 安全辅助函数

7.16. 表情辅助函数

7.17. 字符串辅助函数

7.18. 文本辅助函数

7.19. 排版辅助函数

7.20. URL 辅助函数

7.21. XML 辅助函数

以下待删除

第八章 快速入门

8.1. 安装

8.1.1. 安装

8.1.2. 服务器要求

需要确认自己的环境满足以下要求：

PHP >= 7.2.5

BCMath PHP 拓展

Ctype PHP 拓展

Fileinfo PHP 拓展

JSON PHP 拓展

Mbstring PHP 拓展

OpenSSL PHP 拓展

PDO PHP 拓展

Tokenizer PHP 拓展

XML PHP 拓展

8.1.2.1. 安装 Laravel

Laravel 使用 Composer 来管理项目依赖。因此，在使用 Laravel 之前，请确保你的机器已经安装了 Composer。

```
composer create-project --prefer-dist laravel/laravel blog
```

8.1.2.2. 配置

公共目录

安装完 Laravel 之后，你必须将 web 服务器根目录指向 public 目录。该目录下的 index.php 文件将作为所有进入应用程序的 HTTP 请求的前端控制器。

配置文件

Laravel 框架的所有配置文件都放在 config 目录中。每个选项都有注释，方便你随时查看文件并熟悉可用的选项。

目录权限

安装完 Laravel 后，你可能需要给这两个文件配置读写权限：storage 目录和 bootstrap/cache。目录应该允许 Web 服务器写入，否则 Laravel 程序将无法运行。

应用密钥

安装 Laravel 之后下一件应该做的事就是将应用程序的密钥设置为随机字符串。通过 Composer 安装的，默认已经设置好了。

如果应用程序密钥没有被设置，就不能确保你的用户会话和其他加密数据的安全！

其他配置

除了以上的配置，Laravel 几乎就不需要再配置什么了。你随时就能开发。但是，可能的话还是希望你查看 config/app.php 文件及其注释。

8.1.3. Web 服务器配置

8.1.3.1. 目录配置

Laravel 应该始终在你的 Web 服务器配置的 “Web 目录”的根目录中使用。

8.1.3.2. 优雅链接

Apache

Laravel 中包含了一个 public/.htaccess 文件，通常用于在资源路径中隐藏 index.php 的前端控制器。在用 Apache 为 Laravel 提供服务之前，确保启用了 mod_rewrite 模块，这样 .htaccess 文件才能被服务器解析。

8.2. 配置信息

8.2.1. 介绍

Laravel 框架的所有配置文件都保存在 config 目录中。每个选项都有说明，你可随时查看这些文件并熟悉都有哪些配置选项可供你使用。

8.2.2. 环境配置

对于应用程序运行的环境来说，不同的环境有不同的配置通常是很有用的。例如，你可能希望在本地使用的缓存驱动不同于生产服务器所使用的缓存驱动。

8.2.2.1. 环境变量类型

.env 文件中的所有变量都被解析为字符串，因此创建了一些保留值以允许

你从 env() 函数中返回更多类型的变量。

.evn 值	env() 值
true	(bool) true
(true)	(bool) true
false	(bool) false
(false)	(bool) false
empty	(sting) ''
(empty)	(sting) ''
null	(null) null
(null)	(null) null

8.2.2.2. 检索环境配置

当应用程序收到请求时，.env 文件中列出的所有变量将被加载到 PHP 的超级全局变量 $_ENV 中。你可以使用 env 函数检索这些变量的值。事实上，如果你查看 Laravel 的配置文件，你就能注意到有数个选项已经使用了这个函数。

```
'debug' => env('APP_DEBUG', false),
```

8.2.2.3. 确定当前环境

应用程序当前所处环境是通过 .env 文件中的 APP_ENV 变量确定的。你可以通过 App façade 中的 environment 方法来访问此值。

```
$environment = App::environment();
```

8.2.2.4. 在调试页面隐藏环境变量

当一个异常未被捕获并且 APP_DEBUG 环境变量为 true 时，调试页面会显示所有的环境变量和内容。在某些情况下你可能想隐藏某些变量。你可以通过设置 config/app.php 配置文件中的 debug_blacklist 选项来完成这个操作。

8.2.3. 访问配置值

你可以轻松地在应用程序的任何位置使用全局 config 函数来访问配置值。配置值的访问可以使用[点]语法，这其中包含了要访问的文件和选项的名称。还可以指定默认值，如果配置选项不存在，则返回默认值。

```
$value = config('app.timezone');
```

8.2.4. 配置缓存

为了给你的应用程序提升速度，你应该使用 Artisan 命令 config:cache 将所有的配置文件缓存到单个文件中。这会把你的应用程序中所有的配置选项合并成一个单一的文件，然后框架会快速加载这个文件。

8.2.5. 维护模式

当应用程序处于维护模式时，所有对应应用程序的请求都显示为一个自定义视图。这样可以在更新或执行维护时轻松地[关闭]你的应用程序。维护模式检查包含在应用程序的默认中间件栈中。

8.3. 文件夹结构

8.3.1. 简介

框架默认的目录结构试图为不管是大型应用还是小型应用提供一个良好的起点。当然，您也可以按照自己的喜好重新组织应用的目录结构，只要 Composer 可以自动加载入它们要即可。

当开始使用 Laravel 时，许多开发人员都因缺少 models 目录而感到困惑。然而，缺少这样的目录是故意的。我们发现模型含混不清，因为不同的人对模型有不同的理解。一些开发者把应用的模型称为所有业务逻辑的总体，而另一些人

将模型称为与关系数据加交互的类。

出于这个原因，我们默认把 Eloquent 的模型放在 app 目录下，并允许开发人员将它们放在其他地方。

8.3.2. 根目录

8.3.2.1. app 目录

app 目录包含了应的核心代码。

8.3.2.2. 启动目录

Bootstrap 目录包含了少许文件.app.php 用于框架的启动和自动载入配置，还有一个 cache 文件夹，里面包含了框架为提升性能所生成的文件，如路由和服务缓存文件。

8.3.2.3. 配置目录

Config 目录包含了应用所有的配置文件。

8.3.2.4. 数据库目录

Database 目录包含了数据填充和迁移文件以及模型工厂类。

8.3.2.5. Public 公开目录

Public 目录包含了应用入口文件 index.php 和前端资源文件。

8.3.2.6. 资源目录

Resources 目录包含了应用视图文件和未编译的原生前端资源文件，以及本地化语言文件。

8.3.2.7. 路由目录

Routes 目录包含了应用定义的所有路由。

8.3.2.8. 文件存储目录

Storage 目录包含了编译后的 blade 模板、基于文件的 Session、文件缓存，以及其他由框架生成的文件，该目录细分为 app、frameworkd 和 logs 子目录。

8.3.2.9. 测试目录

Tests 目录包含自动化测试文件，其中默认已经提供了一个开箱即用的 PHPUnit 参考。

8.3.2.10.Vendor 目录

Vendor 目录包含了应用所有通过 Composer 加载的依赖。

8.3.3. 应用目录

App 目录个包含额外的各种目录，如 Console、Http、Providers 等。Console 和 Http 目录提供了进入应用核心的 API，HTTP 协议和 CLI 是和应用进行交互的两种机制，但实际上并不包含应用逻辑。换句话说，它们只是两个向应用发送命令的方式。

8.3.3.1. Broadcasting 目录

Broadcating 目录包含应用程序的所有广播通道类。这些类使用 make:channel 命令生成。此目录默认是不存在的，但是当你创建第一个通道时它将被创建。

8.3.3.2. Console 目录

Console 目录包含应用程序的所有自定义 Artisan 命令。这些命令可以使用 make:command 命令生成。

8.3.3.3. Events 目录

默认情况下这个目录是不存在的。

8.3.3.4. Exceptions 目录

Exceptions 目录包含应用程序的异常处理，并且也是一个放置应用程序抛出任何异常的好地方。

8.3.3.5. Http 目录

Http 目录包含你的控制器，中间件和表单请求。处理进入应用程序请求的所有逻辑几乎都放置在些目录。

8.3.3.6. Job 目录

默认情况下这个目录是不存在的。

8.3.3.7. Listeners 目录

默认情况下这个目录是不存在的。

8.3.3.8. Mail 目录

默认情况下这个目录是不存在的。

8.3.3.9. Notifications 目录

默认情况下这个目录是不存在的。

8.3.3.10.Policies 目录

默认情况下这个目录是不存在的。

8.3.3.11.Providers 目录

Providers 目录包含应用程序的服务提供者。服务提供者通过在服务容器中绑定服务引导应用程序，注册事件或者准备为应用程序即将到来的请求执行其它任何任务。

8.3.3.12.Rules 目录

默认情况下这个目录是不存在的，但如果执行 make:rule Artisan 命令，它将被创建出来。

8.4. 部署

8.4.1. 简介

当您准备部署应用到生产环境时，有一些重要的事情可以确保应用尽可能高效地运行。

8.4.2. 服务器配置

如果你想要部署你的应用到 Nginx 服务器上，你可能会用到下面这个配置文件作为一个开始来配置你的 Web 服务器。

8.4.3. 改进

当你准备往生产环境部署应用时，确保你优化了你的 Composer 类的自动加载映射，这样可以使 Composer 可以很快的找到正确的加载文件去加载给定的类。

第九章 补充说明

暂无。